Synthèse sur les opérations de mise à jour

1. Les contraintes définies au niveau de la base de données

Ce sont les contraintes les plus importantes car elles se trouvent au plus près de la donnée et ne peuvent pas être contournées, contrairement aux autres.

On distingue les contraintes s'appliquant sur une colonne d'une table ou sur une table

Not null, default, check, primary key, unique, foreign key

Lorsque ces contraintes SQL, ne suffisent plus à exprimer une règle métier complexe, il est nécessaire de recourir aux **déclencheurs**, aux **procédures stockées** ou aux **transactions**.

Un déclencheur (trigger) est un objet SQL qui exécute automatiquement du code lorsqu'un événement (insertion, modification, suppression) se produit sur une table (ou parfois une vue).

Un trigger peut s'exécuter avant l'opération ce qui permet de faire des contrôles préalables, de modifier les données et éventuellement empêcher l'opération.

Un trigger peut s'exécuter après l'opération, dans ce cas il s'agit souvent de mettre à jour d'autres données ou d'autres tables.

Avantages d'un déclencheur

- Centralisation des règles de validité dans la base elle-même, quelle que soit l'application (site web, back-office, phpMyAdmin…)
- Messages fonctionnels personnalisés, retournés directement par le déclencheur
- Garantie d'un respect systématique des règles, indépendamment de l'origine de la requête SQL

Inconvénients d'un déclencheur

- Opacité du code métier : les règles sont implémentées "dans la base", donc moins visibles pour les développeurs. Cela peut rendre le débogage plus complexe.
- En cas de changement de règle métier, il faut modifier le code SQL du déclencheur, ce qui implique une bonne maîtrise de SQL procédural.
- Les performances peuvent être dégradées si le déclencheur effectue des requêtes coûteuses sur des tables volumineuses (ex. : détection de chevauchements dans une grande table de catégories).

L'utilisation d'un déclencheur permet donc de renforcer la robustesse fonctionnelle de la base en y inscrivant des règles métier avancées.

Il permet également de déléguer à la base la responsabilité de vérifier la cohérence des données tout en fournissant à l'utilisateur des messages clairs et pertinents.

Cependant, elle implique une certaine discipline de documentation et une vigilance accrue pour les développeurs qui interagissent avec la base.

2. La classe technique Table

La classe Table constitue le cœur technique de l'accès aux données de l'application.
Son objectif est de centraliser toutes les opérations communes aux différentes tables afin d'éviter la répétition de code dans les classes filles.

Elle prend en charge :
- la connexion à la base de données ;
- la gestion des colonnes et des objets `Input` associés ;
- la validation générique des données ;
- la gestion des erreurs ;
- le contrôle des contraintes d'unicité ;
- l'insertion des enregistrements ;
- la modification des enregistrements ;
- la suppression des enregistrements.

Développement Web > Eléments pratiques > Classe technique > table

Les classes dérivées doivent uniquement définir les éléments propres à leur table et leurs règles métier.

3. Les classes métier dérivées de la classe Table

Une classe métier représente une table de la base de données.
Elle encapsule toutes les règles propres à cette table :

- le nom de la table, la clé primaire et les colonnes (méthode configure)
- les règles métier spécifique (méhodes beforeInsert(), beforeUpdate() et beforeDelete())

Gestion des valeurs calculées automatiquement

Lorsqu'une colonne obtient sa valeur à partir d'une règle de calcul ou d'une information externe, cette valeur doit être affectée avant l'exécution de l'insertion ou de la modification.

Cette logique doit être implémentée dans :
- beforeInsert() pour une création ;
- beforeUpdate() pour une modification.

La valeur est affectée à l'aide de la méthode : setValue()
Exemple :

```
protected function beforeInsert(): bool {
    $this->setValue("idUtilisateur", $_SESSION['idUtilisateur'] );
    return true;
}
```

La classe métier reste ainsi responsable uniquement des règles fonctionnelles, tandis que la classe Table conserve la responsabilité technique de l'accès aux données.

Important :

Les colonnes dont la valeur est obtenue automatiquement depuis la base de données (clause DEFAULT ou via un déclencheur) ne doivent pas être déclarées dans la classe métier

Le recours à la classe Table n'est pas possible dans les cas suivants :

La table possède une clé primaire composée (composite)
L'opération de mise à jour doit être encapsulée dans une transaction complexe impliquant plusieurs tables ou étapes critiques (ex. cohérence entre plusieurs insertions).

Développement Web > Eléments pratiques > Classe métier

4. Les objets Column et la validation des colonnes

La classe Table utilise des objets issus d'une classe dérivée de la classe Column pour représenter les colonnes d'une table et assurer automatiquement la validation des données reçues.

La classe héritée est choisie en fonction du type de donnée attendu : ColumnInt pour un entier, ColumnTexte pour une valeur alphanumérique, ColumnDate pour un date, ColumnEmail pour une adresse mail, ColumnURL pour une url, ColumnListe pour une zone de liste, etc.

Le contrôle des données est ainsi délégué aux classes Column.

La classe Column constitue la classe de base de tous les contrôles de données.
Elle assure le contrôle commun à toutes les colonnes :

- vérification de la présence d'une valeur obligatoire ;
- stockage et **nettoyage** de la valeur (afin d'éviter toute injection) ;
- stockage du message de validation.

Propriétés principales :

```
public mixed $Value = null;
public bool $Require = true;
```

Méthode principale : checkValidity() : bool

Chaque classe dérivée implémente la méthode checkValidity(): bool
Cette méthode retourne :

- true si la valeur est correcte ;
- false si une erreur est détectée.

En cas d'erreur, le message est accessible avec la méthode getValidationMessage()

Les principales classes dérivées

- **ColumnText :** Contrôle une chaîne de caractères.

Elle permet notamment de gérer :
- la longueur minimale : $MinLength;
- la longueur maximale : $MaxLength ;
- une expression régulière : $Pattern *('/^[A-Z]+$/u) ;*
- la gestion de la casse : $Casse (Upper, Lower, Word, First)
- la suppression des accents : $SupprimerAccent ;
- la suppression des espaces superflus : $SupprimerEspaceSuperflu

Exemple :

```
$input = new ColumnText();
$Column->Require = true;
$Column->Casse = TextCase::Upper;
$Column->SupprimerAccent = true;
$Column->SupprimerEspaceSuperflu = true;
$Column->Pattern = '^[A-Z]( ?[A-Z]+)*$';
$Column->MaxLength = 30;
$Column->Casse = TextCase::First;
```

- **ColumnInt :** Contrôle une valeur entière.

Il permet de définir une valeur minimale et une valeur maximale.

Exemple :

```
$age = new ColumnInt();
$age->Min = 18;
$age->Max = 99;
```

- **ColumnList :** Contrôle une valeur appartenant à une liste autorisée.

Exemple :

```
$sexe = new ColumnList();
$sexe->setValues([ "M", "F" ]);
```

- **ColumnDate :** permet de contrôler la validité d'une date au format aaa-mm-jj en fixant éventuellement un intervalle à l'aide des propriétés $Min et $Max

Exemple :

```
$Column = new ColumnDate();
$Column->Require = true;
$Column->Min = Categorie::getDateNaissanceMin();
$Column->Max = Categorie::getDateNaissanceMax();
```

- **ColumnMail :** permet de vérifier la validité du format de l'adresse et l'existence du domaine
- **ColumnUrl :** permet de contrôler la validité de l'url .

Lors d'une opération add() ou modify(), la méthode checkColumns() de la classe Table :

- récupère la valeur transmise ;
- l'affecte à l'objet Column associé ;
- exécute la méthode sanityze pour nettoyer éventuellement des balises malfaisantes intégrées dans la valeur
- exécute la méthode checkValidity() sur cet objet ;
- récupère le message d'erreur en cas d'échec.

Avantage de cette architecture

Grâce aux objets Column :
- les règles de validation sont regroupées avec la colonne concernée ;
- les classes métier restent simples ;
- les contrôleurs n'ont pas besoin de connaître les règles de validation ;
- les messages d'erreur sont uniformisés.

La classe Table orchestre la validation, tandis que chaque objet Column connaît la manière de contrôler sa propre valeur.

Développement Web > Eléments pratiques > Les classes Column

5. Les scripts CRUD

5.1. Modèle pour le contrôleur d'ajout

Le script d'ajout reçoit une requête HTTP contenant :

- les valeurs des colonnes à enregistrer (`columns`).

Dans l'application actuelle, ces données sont transmises dans une requête POST.
Elles peuvent être envoyées sous forme de paramètres de formulaire ou dans le corps d'une requête JSON.

Le script utilise la classe `Requete` pour récupérer et contrôler les données reçues, puis crée automatiquement l'objet métier associé grâce à `FabriqueTable`.

Il appelle ensuite la méthode `add()` de cet objet métier qi prend en charge.

Le contrôle des valeurs transmises : checkColumns() ; Le contrôle des contraintes d'unicité : checkUniqueConstraints() ; Les contrôles spécifiques à la classe métier : beforeInsert() ; L'insertion dans la base de données : insert().

Exemple

```
use ClasseMetier\Annonce;

use ClasseTechnique\ReponseJson;
use ClasseTechnique\Requete;
use ClasseTechnique\Std;

/** @noinspection PhpIncludeInspection */
require $_SERVER['DOCUMENT_ROOT'] . '/../bootstrap/bootstrap.php';

// Vérification d'un appel AJAX par la méthode POST sécurisé par un jeton
Requete::exigerPost();

// récupération des données transmises
$columns = Requete::postArray('columns');

// création de l'objet métier
$annonce = new Annonce();

// ajout de l'annonce
$resultat = $annonce->add($columns);

if ($resultat === true) {
    ReponseJson::envoyerMessage("Annonce ajoutée");
}

// en cas d'erreur
ReponseJson::envoyerLesErreurs($annonce->getErrors());
```

5.2. Script générique de modification

Le script de modification reçoit une requête HTTP contenant :

- l'identifiant de l'enregistrement à modifier (`primaryKey`) ;
- les nouvelles valeurs des colonnes à modifier (`columns`).

Contrairement à un ajout, une modification nécessite obligatoirement de connaître l'enregistrement concerné. La clé primaire est donc transmise séparément des valeurs à modifier.

Exemple :

```
{
"primaryKey" : "000001",
"columns" :
 {
    "nom" : "Legrand",
    "prenom" : "pierre",
    "dateNaissance" : "2010-01-01",
    "sexe" : "M",
    "idClub" : "080021"
 }
}
```

La méthode modify() de la classe Table prend alors en charge :

la vérification de l'existence de l'enregistrement identifié par la clé primaire ;
le contrôle des colonnes autorisées en modification ;
la validation des nouvelles valeurs avec les objets Column ;
le contrôle des contraintes d'unicité ;
l'application éventuelle des règles métier spécifiques ;
la génération et l'exécution de la requête SQL UPDATE.

La classe métier peut intervenir avant la modification en redéfinissant la méthode protected function beforeUpdate(mixed $id): bool

Cette méthode permet notamment :

de vérifier une cohérence entre plusieurs colonnes ;
de calculer une valeur automatiquement ;
d'empêcher une modification selon une règle métier.

Le script modifier.php reste donc générique, il ne connaît ni la structure de la table, ni les colonnes modifiables, ni les règles métier.

Ces responsabilités sont entièrement déléguées à la classe métier et à la classe technique Table.

Exemple :

```
use ClasseMetier\Coureur;

use ClasseTechnique\ReponseJson;
use ClasseTechnique\Requete;

/** @noinspection PhpIncludeInspection */
require $_SERVER['DOCUMENT_ROOT'] . '/../bootstrap/bootstrap.php';

// Vérification d'un appel AJAX par la méthode POST sécurisé par un jeton
// Requete::exigerPost();

// récupération des données transmises
$licence = Requete::postString('primaryKey');
$columns = Requete::postArray('columns');

// création de l'objet métier
$coureur = new Coureur();

// modification du coureur
$resultat = $coureur->modify($licence, $columns);


if ($resultat === true) {
    ReponseJson::envoyerMessage("Coureur modifié");
}

// en cas d'erreur
ReponseJson::envoyerLesErreurs($coureur->getErrors());
```

5.3. Modèle pour le contrôleur de suppression

Le script ajax/supprimer.php reçoit une requête HTTP contenant l'identifiant de l'enregistrement à supprimer (`primaryKey`).

Exemple :

```
{  "primaryKey": "123456"}
```

La suppression repose uniquement sur l'identification précise de l'enregistrement concerné.
Le script utilise la classe Requete afin de récupérer et contrôler les données transmises :

La méthode delete() de la classe Table prend en charge :

- la vérification de l'existence de l'enregistrement ;
- l'application éventuelle des règles métier avant suppression ;
- l'exécution de la requête SQL DELETE.

Avant la suppression effective, la classe métier peut intervenir en redéfinissant la méthode protected function beforeDelete(mixed $id): bool. Cette méthode permet notamment :

- d'interdire une suppression si des dépendances existent ;
- de vérifier qu'une règle métier autorise la suppression ;
- d'effectuer éventuellement des traitements préparatoires.

Exemple concret : Une catégorie peut être supprimée uniquement si aucun coureur n'est encore associé.

La classe métier peut donc empêcher la suppression
Il délègue entièrement ces responsabilités à la classe métier et à la classe technique Table.

Exemple

```
// Vérification d'un appel AJAX par la méthode POST sécurisé par un jeton
Requete::exigerPost();

// récupération des données transmises
$licence = Requete::postString('primaryKey');

// création de l'objet métier
$Coureur = new Coureur();

// suppression d'un coureur
$resultat = $Coureur->delete($licence);

if ($resultat === true) {
   ReponseJson::envoyerMessage("Coureur supprimé");
}

// en cas d'erreur
ReponseJson::envoyerLesErreurs($Coureur->getErrors());
```

6. Appel Ajax des contrôleurs

L'utilisation d'AJAX permet de transmettre les données du formulaire sans rechargement complet de la page.

Le code JavaScript ne connaît pas les requêtes SQL ni la structure interne des classes métier. Il transmet uniquement :

les valeurs des colonnes nécessaires ;
éventuellement la clé primaire permettant d'identifier un enregistrement existant.

Les scripts PHP prennent ensuite en charge la création de l'objet métier adapté et délèguent le traitement aux méthodes de la classe Table.

6.1. Ajout d'un enregistrement

L'ajout consiste à construire un objet JavaScript contenant les colonnes à enregistrer puis à appeler le script ajouter.php.

Exemple :

```
function ajouter() {
    const columns = {
        licence: licence.value,
        nom: nom.value,
        prenom: prenom.value,
        sexe: sexe.value,
        dateNaissance: dateNaissance.value,
        idClub: idClub.value,
        ffa: ffa.checked ? 1 : 0
    };

    // prise en compte des champs optionnels
    if (email.value.length > 0) {
        columns.email = email.value;
    }
    if (telephone.value.length > 0) {
        columns.telephone = telephone.value;
    }

    //  demande d'ajout dans la base de données
    appelAjax({
        url: 'ajax/ajouter.php',
        data: {
            columns: columns
        },
        success: (data) => {
                retournerVersApresConfirmation(data.message, '/coureur/liste');
        }
    });
}}
```

Les données envoyées au serveur correspondent à :

```
{
   "columns": {
      "licence": "123456",
      "nom": "DUPONT",
      "prenom": "Jean",
      "sexe": "M",
      "dateNaissance": "2010-05-15",
      "idClub": "10",
      "ffa": "1"
   }
}
```

Les colonnes non fournies par JavaScript peuvent être complétées automatiquement par :
- la base de données ;
- la méthode beforeInsert() de la classe métier.

6.2. Modification d'un enregistrement

Une modification nécessite une information supplémentaire : la clé primaire de l'enregistrement concerné

Exemple :

```
function modifier() {
   msg.innerHTML = '';

   const columns = {
      nom: nom.value,
      prenom: prenom.value,
      sexe: sexe.value,
      dateNaissance: dateNaissance.value,
      idClub: normaliserValeur(idClub.value),
      ffa: ffa.checked ? 1 : 0,
      email: email.value.trim().length > 0 ? email.value : null,
      telephone: telephone.value.trim().length > 0 ? telephone.value : null
   };

   appelAjax({
      url: 'ajax/modifier.php',
      data: {
         primaryKey: coureur.licence,
         columns: columns
      },
      success: () => {
         // il faut remplacer les anciennes valeurs par les nouvelles au niveau de l'objet coureur
         coureur.nom = nom.value;
         coureur.prenom = prenom.value;
         …
         afficherToast('Coureur modifié');
      }
   });
}}
```

Les données envoyées sont de la forme :

```
{
   "primaryKey": "123456",
   "columns": {
      "nom": "MARTIN",
      "prenom": "Jean",
      "sexe": "M",
      …
   }
}
```

La clé primaire est séparée des colonnes modifiées car elle ne constitue pas une donnée à modifier.

Une modification peut être partielle :

```
{
   "columns": {
      "email" : "jean.martin@saint-remi.net",
      "telephone": "0600000000"
   }
}
```

Seule les colonnes telephone et email seront modifiées.

6.3. Suppression d'un enregistrement

La suppression nécessite uniquement l'identification de l'enregistrement et le nom de la tablet.

Exemple :

```
function supprimer() {
   appelAjax({
      url: 'ajax/supprimer.php',
      data: {
         primaryKey: coureur.licence
      },
      success: () => {
         afficherToast('Coureur supprimé');
         search.focus();
      }
   });
}
```

Les données envoyées sont :

```
{
   "primaryKey": "123456"
}
```

Synthèse sur les opérations de mise à jour

7. Synthèse

Pour gérer les opérations de mise à jour sur une table MySQL, il faut :

- créer une nouvelle classe métier héritant de Table
- déclarer ses paramètres (nom de la table, nom de la clé primaire et colonnes) dans la méthode configure
- implémenter si besoin les méthodes beforeInsert, beforeUpate et beforeDelete
- ajouter les contraintes de cette classe dans le fichier config/contrainte.php (les contraintes check ne seront pas exploitées si la configuration des colonnes est bien réalisée)
- créer les contrôleurs assurant l'ajout, la modification et la suppression selon les modèles fournis
- réaliser côté client les appels Ajax vers ces contrôleurs